**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6 Дисциплина:** Архитектура ЭВМ

**Тема:** Арифметические операции в NASM.

**Студент: А н ь с я о я н ь**

**Группа:** НПИ-01-25

**Студенческий билет: 1 1 3 2 2 5 8 1 9 1**

**Преподаватель:** Демидова А. В.

**1. Цель работы** Освоение арифметических инструкций языка ассемблера NASM. Приобретение навыков работы с числами и преобразования их из символьного вида.

**2. Выполнение работы**

**2.1. Подготовка рабочего пространства**
Я создал каталог для шестой лабораторной работы и скопировал библиотеку `in_out.asm`.
**Команда:** `mkdir ~/work/arch-pc/lab06`
**Команда:** `cp ~/work/arch-pc/lab05/in_out.asm ~/work/arch-pc/lab06`
**Результат:** Файлы подготовлены для выполнения заданий.

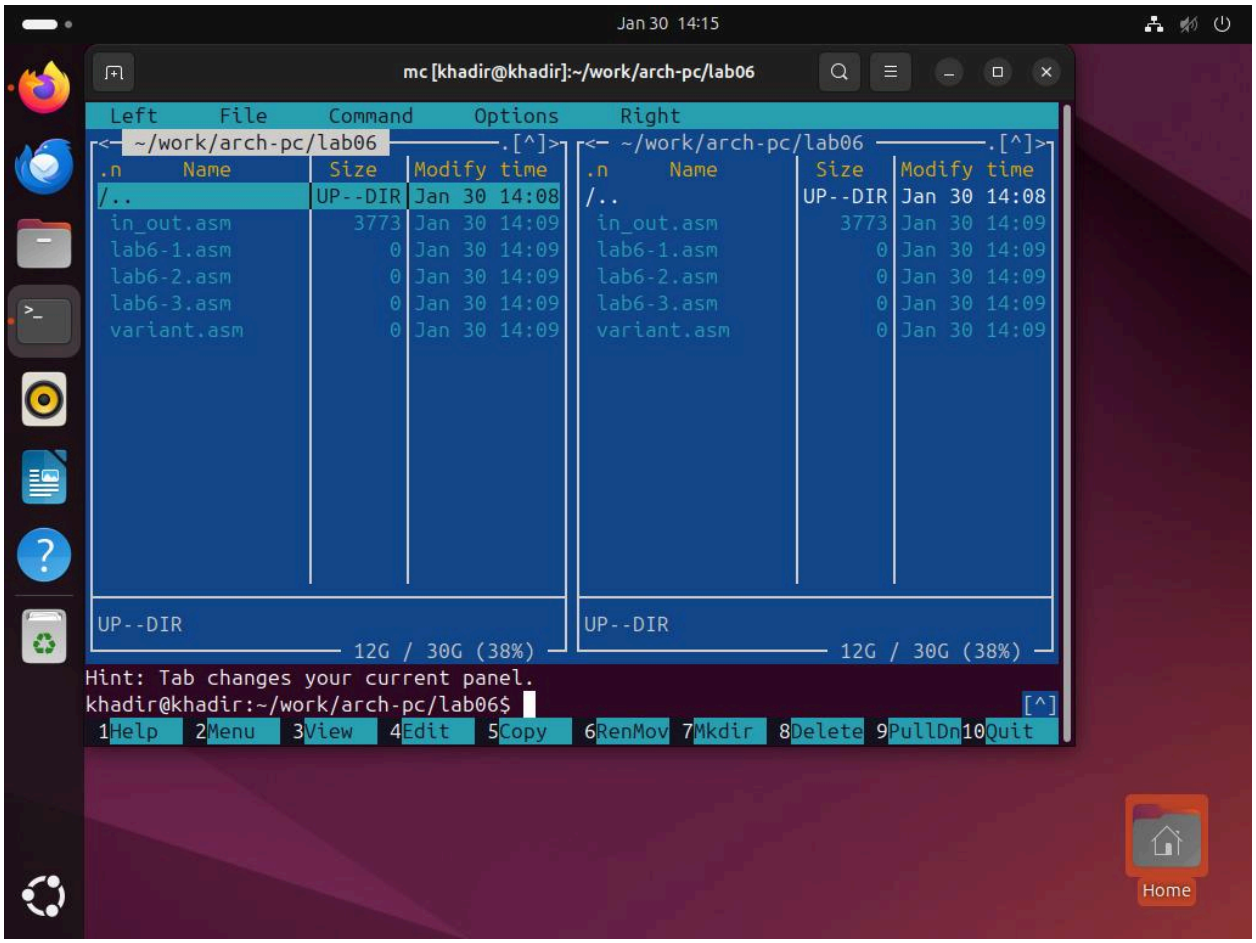


Рис. 6.1. Подготовка рабочего каталога.

**2.2. Программы вывода символьных и численных значений (Листинг 6.1 и 6.2)** Я создал файлы и изучил разницу между выводом символов и чисел. В первой программе сложение кодов '6' и '4' дало букву 'j'. Во второй программе я использовал `iprintLF` для вывода числа. **Трансляция:** `nasm -f elf lab6-1.asm -o lab6-1.o` **Компоновка:** `i686-linux-gnu-ld -m elf_i386 lab6-1.o -o lab6-1` **Результат:** Программы работают корректно.

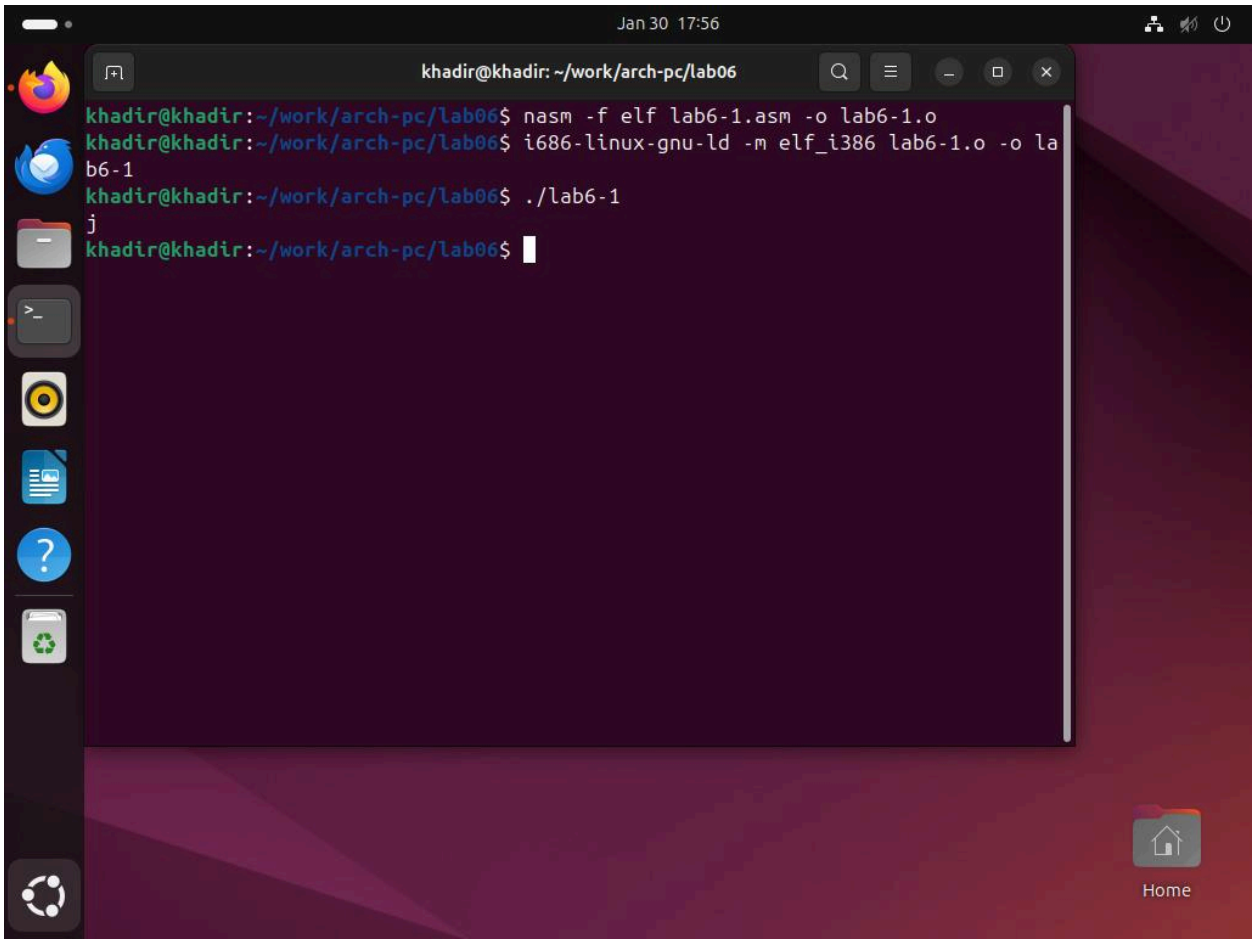


Рис. 6.2. Компиляция и запуск программы lab6-1.asm с символьными данными.

Я изменил программу, заменив символы '6' и '4' на числа 6 и 4. После повторной компиляции программа складывает числа и получает результат 10. В таблице ASCII символ с кодом 10 — это перевод строки (LF), поэтому на экране ничего не отображается.

Трансляция: `nasm -f elf lab6-1.asm -o lab6-1.o`

Компоновка: `i686-linux-gnu-ld -m elf_i386 lab6-1.o -o lab6-1`

Результат: Вывод пустой, так как код 10 — невидимый символ.

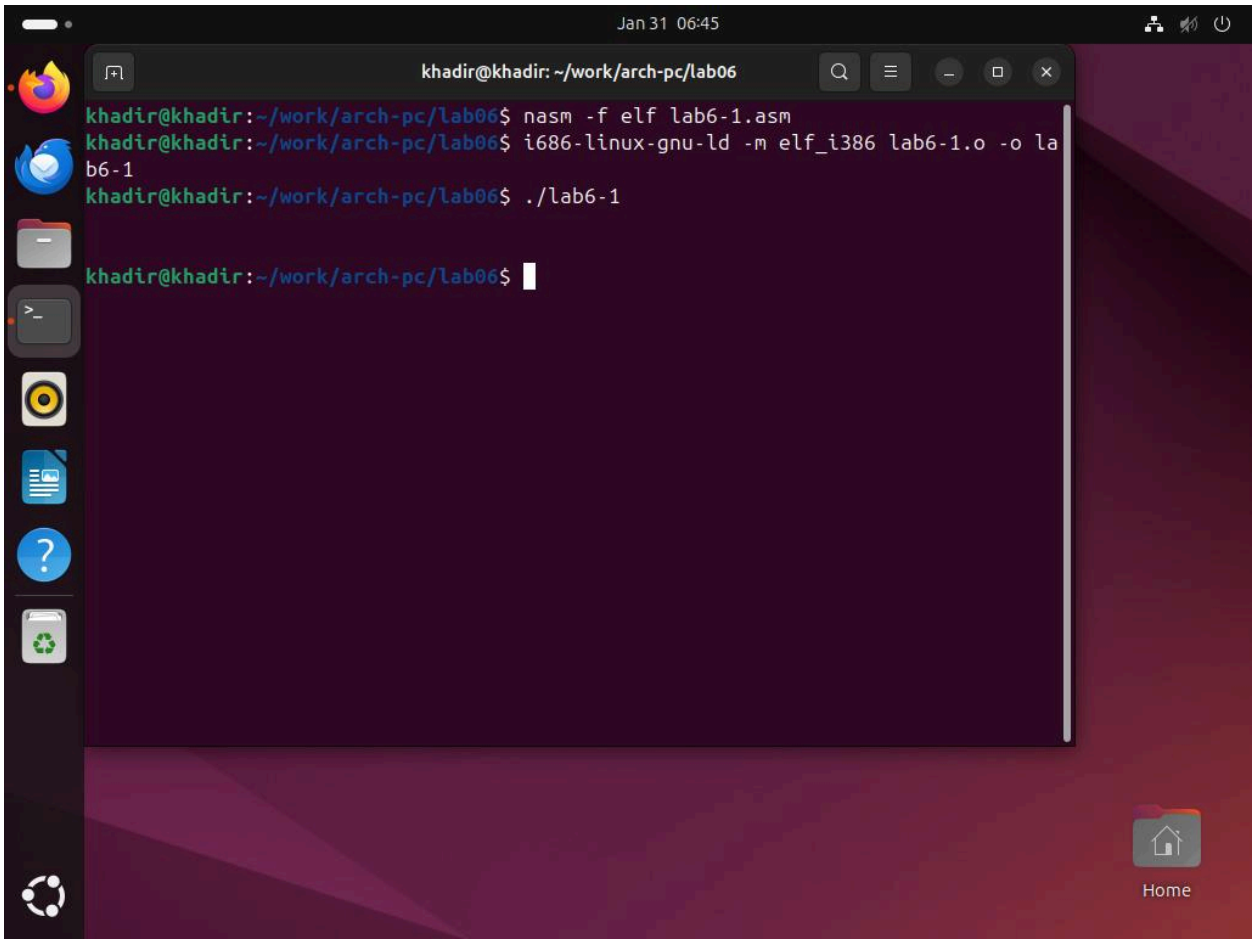


Рис. 6.3. Результат работы программы lab6-1.asm с численными данными.

Я создал файл `lab6-2.asm` и ввел текст программы, которая использует функцию `iprintLF`. В этой программе в регистры по-прежнему записываются символы в кавычках (**'6'** и **'4'**). Процессор складывает их внутренние коды по таблице ASCII (54 и 52 соответственно), получая сумму 106. Благодаря функции `iprintLF`, этот результат выводится на экран как число, а не как символ.

Трансляция: `nasm -f elf lab6-2.asm -o lab6-2.o`

Компоновка: `i686-linux-gnu-ld -m elf_i386 lab6-2.o -o lab6-2`

Запуск: `./lab6-2`

Результат: `106`

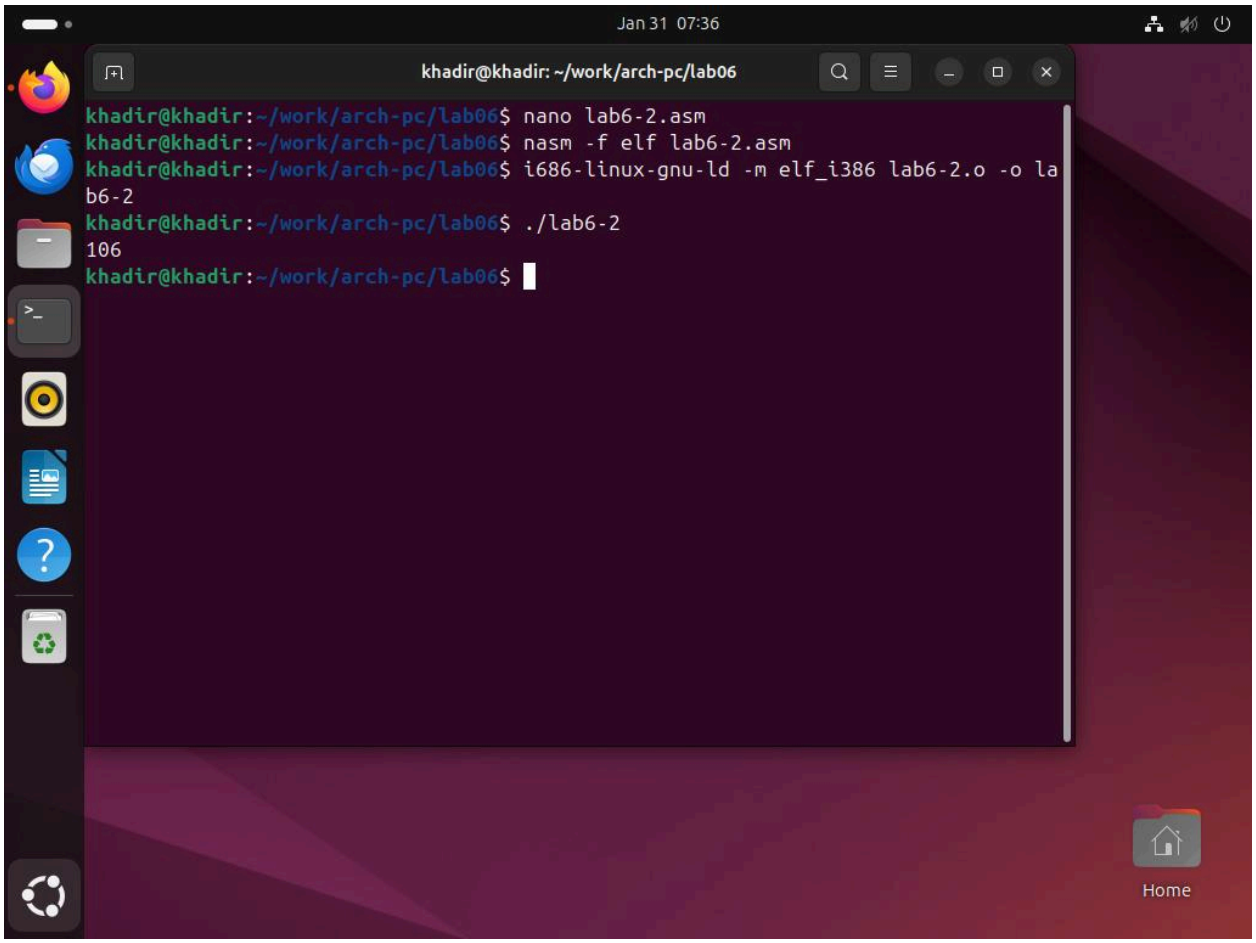


Рис. 6.4. Вывод численного значения суммы ASCII-кодов символов '6' и '4'.

### 2.3. Выполнение арифметических операций

Я создал файл `lab6-3.asm` для вычисления арифметического выражения f(x)=(5∗2+3)/3. В программе использовались инструкции `mul` для умножения и `div` для деления.

Важным этапом перед делением является использование команды `xor edx, edx`. Поскольку делитель является 32-битным регистром, процессор ожидает, что делимое находится в паре регистров **EDX:EAX**. Обнуление **EDX** необходимо для корректного формирования 64-битного значения и предотвращения ошибок в вычислениях.

Трансляция: `nasm -f elf lab6-3.asm -o lab6-3.o`

Компоновка: `i686-linux-gnu-ld -m elf_i386 lab6-3.o -o lab6-3`

Запуск: `./lab6-3`

Результат: `Результат: 4`, `Остаток от деления: 1`

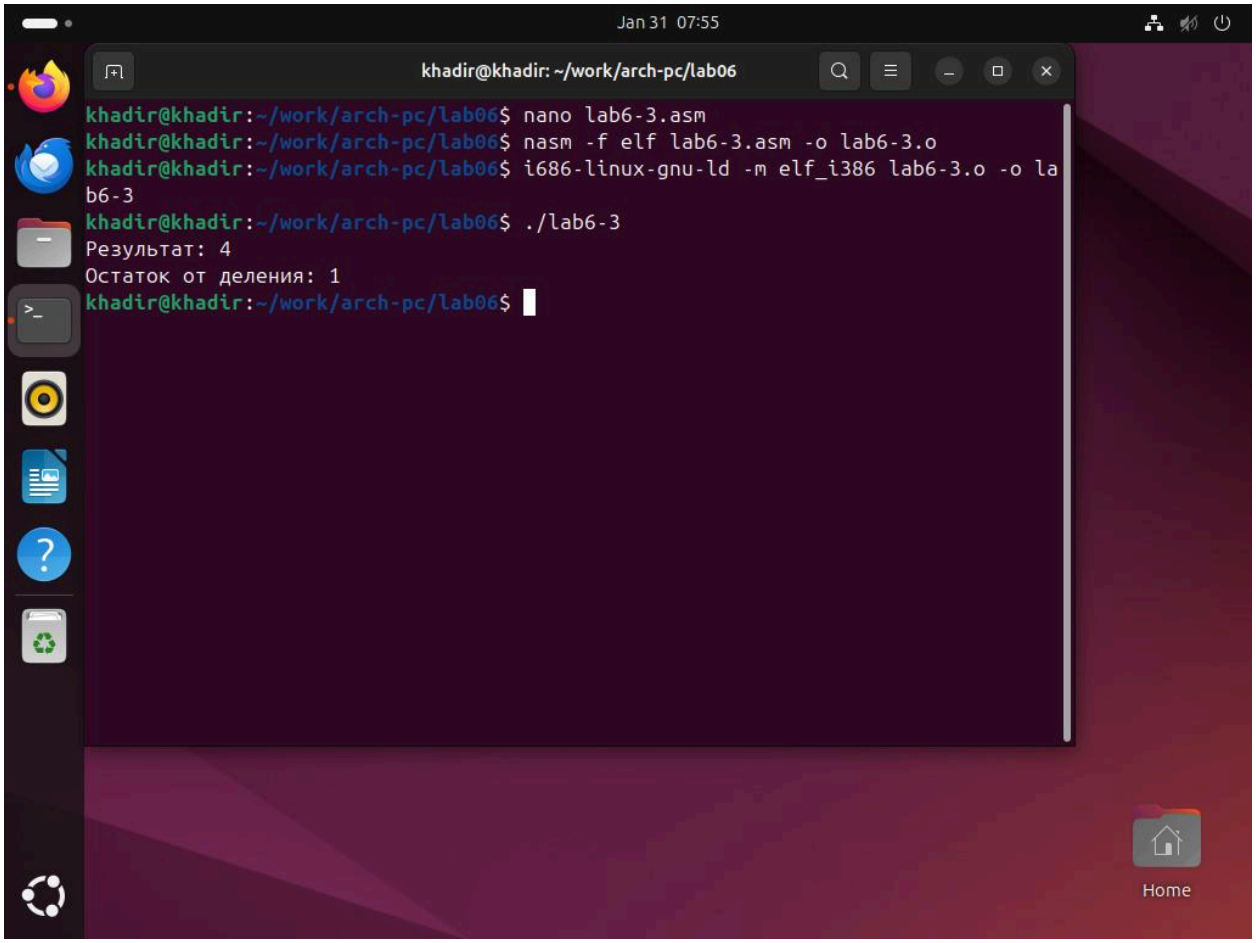


**Рис. 6.5. Результат работы программы вычисления арифметического выражения.**

## 2.4. Вычисление варианта задания

Я написал программу для определения номера своего варианта на основе номера студенческого билета (1132258191). Номер варианта вычисляется по формуле (Snmod20)+1.

Так как данные, вводимые с клавиатуры, воспринимаются как строка символов, я использовал функцию `atoi` из библиотеки **in_out.asm** для преобразования ASCII-кода в целое число перед выполнением арифметических операций.

Трансляция: `nasm -f elf variant.asm -o variant.o`

Компоновка: `i686-linux-gnu-ld -m elf_i386 variant.o -o variant`

Запуск: `./variant`

Ввод: 1 1 3 2 2 5 8 1 9 1 езультат: `Ваш вариант: 12`

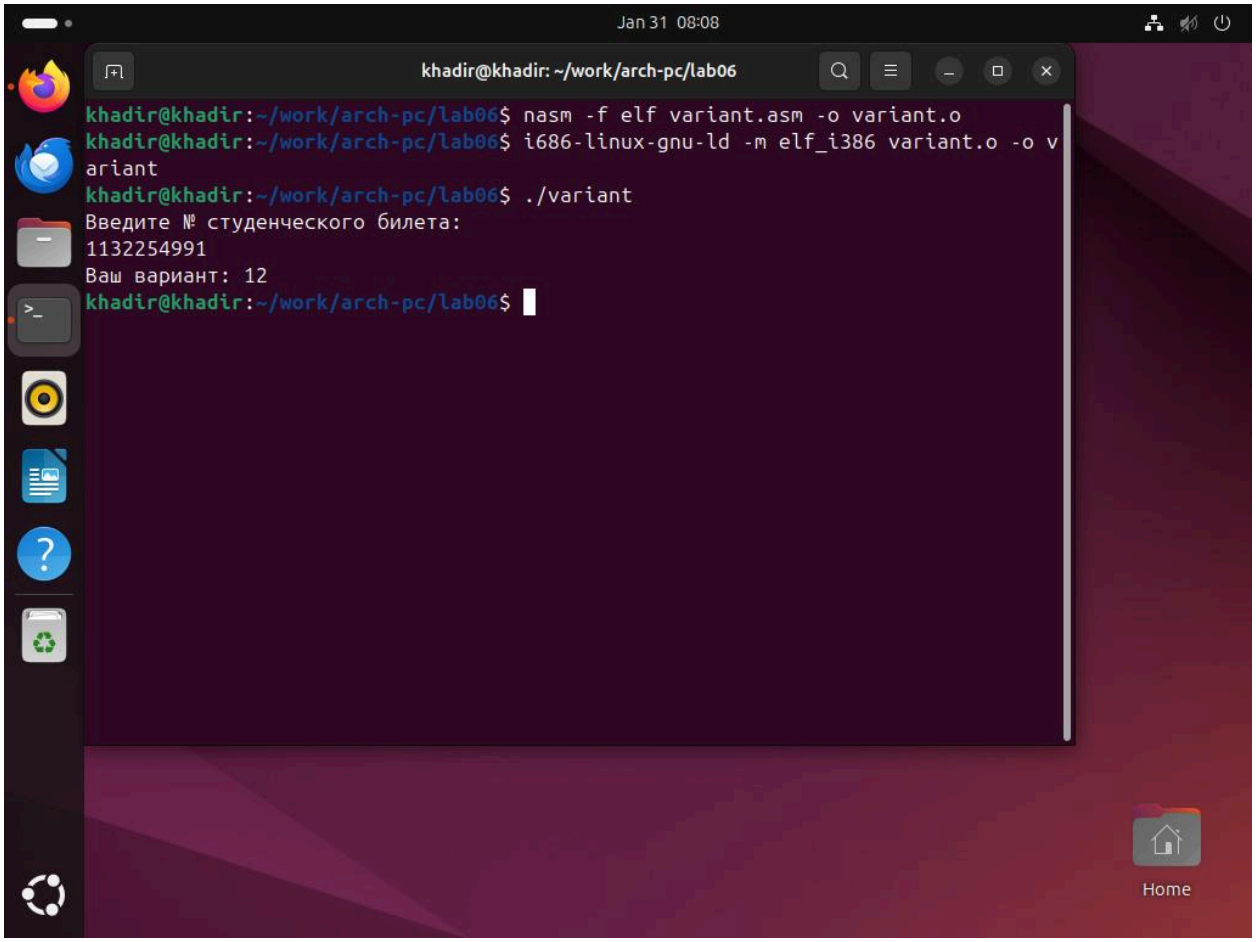


**Рис. 6.6. Определение номера варианта на основе студенческого билета в терминале.**

## 3. Самостоятельная работа (Вариант 12)

Для выполнения самостоятельной работы я использовал вариант №12. Задание заключается в написании программы для вычисления значения функции f(x)=(8x−6)/2.

**Ход работы:**

1. Я создал файл `task.asm` и реализовал ввод значения x с клавиатуры.
2. Использовал функцию `atoi` для преобразования введенной строки в числовое значение.
3. Реализовал арифметический алгоритм:
    - Умножение регистра **EAX** на 8 с помощью инструкции `mul`.
    - Вычитание 6 из полученного результата инструкцией `sub`.

- Деление на 2 с помощью `div`. Перед делением я обнулил регистр **EDX** (`xor edx, edx`), чтобы избежать ошибок при работе с 64-битным делимым **EDX:EAX**.

4. Вывел итоговый результат на экран с помощью `iprintLF`.

**Текст программы (Листинг):**

```
; Вариант 12

; Программа для вычисления функции f(x) = (8x - 6) / 2


%include 'in_out.asm'


SECTION .data

    msg: DB 'Введите x: ',0

    rem: DB 'Результат: ',0


SECTION .bss

    x: RESB 80


SECTION .text

    GLOBAL _start

_start:

    mov eax, msg

    call sprint


    mov ecx, x

    mov edx, 80

    call sread
```

```
mov eax, x
call atoi

mov ebx, 8
mul ebx

sub eax, 6

mov ebx, 2
xor edx, edx
div ebx

mov edi, eax

mov eax, rem
call sprint

mov eax, edi
call iprintLF

call quit
```

**Компиляция и запуск:** Трансляция: `nasm -f elf task.asm -o task.o`

Компоновка: `i686-linux-gnu-ld -m elf_i386 task.o -o task`

Запуск: `./task`

**Проверка корректности:**

- При x=1: (8×1−6)/2=1. Программа вывела: 1.
- При x=5: (8×5−6)/2=17. Программа вывела: 17.

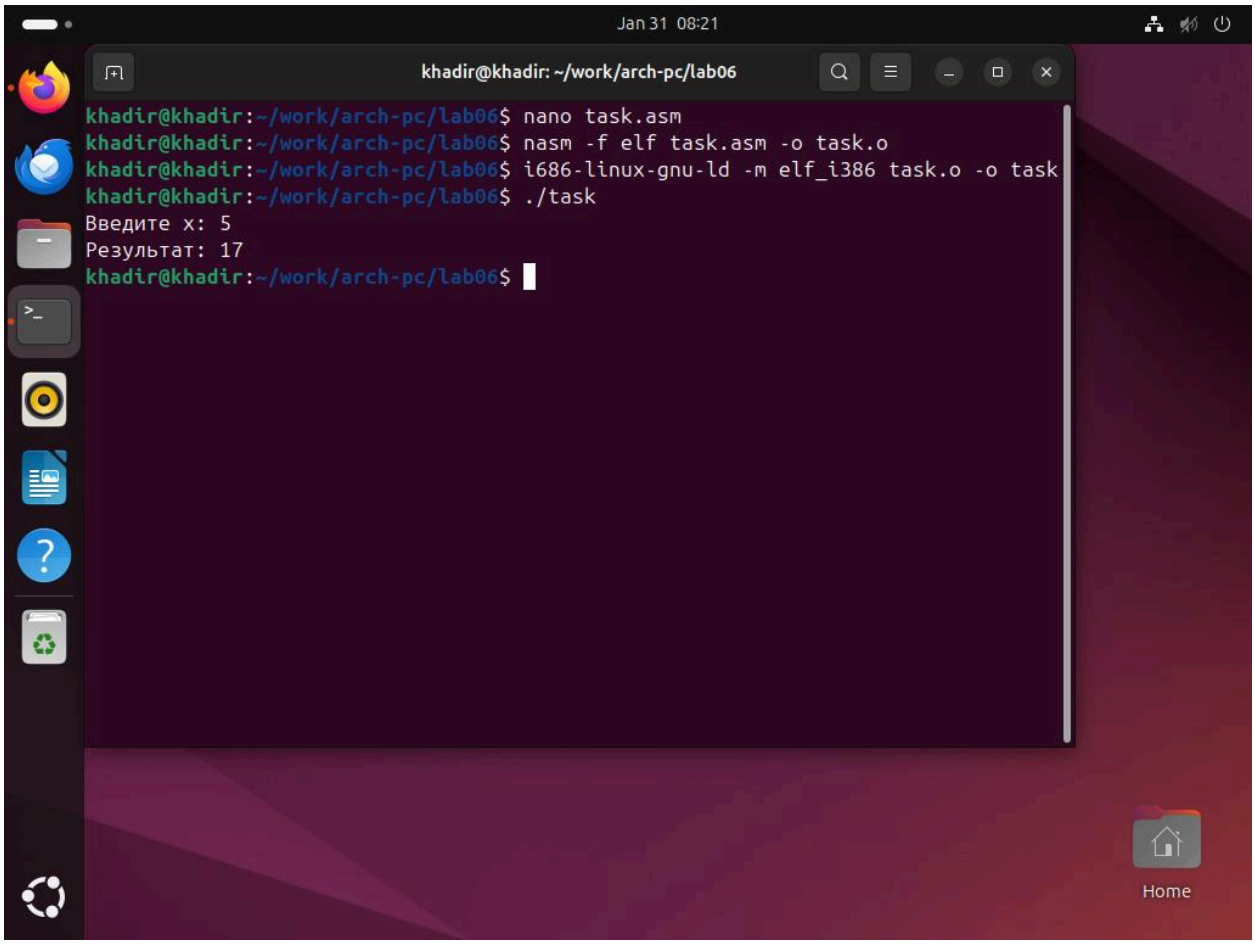


**Рис. 6.7. Результат работы программы и проверка правильности вычислений.**

## 4. Ответы на контрольные вопросы

1. **Как вывести «Ваш вариант»?** Нужно положить адрес строки в `EAX` и вызвать `sprint`.
2. **Зачем нужна строка `mov ecx, x`?** Она записывает адрес переменной `x` в регистр `ECX`, чтобы компьютер знал, куда сохранить ввод.
3. **Зачем нужен `call atoi`?** Чтобы превратить текст (символы) в обычное число для математики.
4. **Как считается вариант?** Командами `div ebx` (деление на 20) и `inc edx` (прибавить 1 к остатку).
5. **Где лежат частное и остаток после `div`?**
   - **EAX** — частное (целое число).
   - **EDX** — остаток.
6. **Зачем нужен `inc edx`?** Чтобы номер варианта был от 1 до 20 (а не от 0 до 19).
7. **Как вывести результат на экран?** Положить число в `EAX` и вызвать `iprintLF`.

## 5. Вывод

В этой работе я научился делать математические операции в Ассемблере. Я узнал, как работают команды `mul` и `div`. Я понял, что компьютер видит ввод как текст, поэтому нужно использовать `atoi`. Также я научился обнулять регистр `EDX` перед делением.